

前三章作业习题讲解

郑智馨

October 21, 2026

Peking University

第一章作业：第 2 题

分析下列程序，求出程序的时间复杂度，用大 O 表示，**给出推导过程**。

```
int cnt = 0;
int i = 1;
while (i <= n) {
    int j = i;
    while (j <= n) {
        cnt++;
        j *= 2;
    }
    i++;
}
```

第一章作业：第 2 题

```
int cnt = 0;
int i = 1;
while (i <= n) {
    int j = i;
    while (j <= n) {
        cnt++;
        j *= 2;
    }
    i++;
}
```

解 对应固定的 i，内层循环执行 $\lfloor \log(n/i) \rfloor + 1$ 次。外层循环枚举 1 到 n 中所有的 i。总的执行次数为：

$$C(n) = \sum_{i=1}^{n} (\lfloor \log(n/i) \rfloor + 1),$$

利用取整的性质进行放缩：

$$\sum_{i=1}^{n} \log(n/i) < C(n) \leq \sum_{i=1}^{n} (\log(n/i) + 1).$$

令：

$$\begin{aligned} S(n) &= \sum_{i=1}^{n} \log(n/i) = \sum_{i=1}^{n} \log n - \sum_{i=1}^{n} \log i \\ &= n \log n - \log(n!) \end{aligned}$$

则：

$$S(n) < C(n) \leq S(n) + n = n \log n - \log(n!) + n.$$

由 Stirling 公式，

$$\log(n!) = n \log n - (\log e)n + O(\log n),$$

所以

$$S(n) = (\log e)n + O(\log n) = \Theta(n), \quad S(n) + n = \Theta(n).$$

因而 $C(n) = \Theta(n)$，答案要求的大 O 表示为 $O(n)$。

第一章作业：第 2 题

主要问题出现在计算 $S(n)+n=n\log n-\log(n!)+n$ 的复杂度上：

1. **直接把两项消掉**：$n\log n-\Theta(n\log n)+n\overset{?}{=}\Theta(n)$。
   e.g. $\Theta(n\log n)$ 实际可能是 $\frac{1}{2}n\log n$，减完还剩 $\frac{1}{2}n\log n$，不能这样推出 $\Theta(n)$。

2. **认为相减后还是同阶**：$n\log n-\Theta(n\log n)\overset{?}{=}\Theta(n\log n)$。

3. **只给出一个上界**：$n\log n-O(n\log n)=O(n\log n)$。
   得到了一个更松的上界。

两个主要项相消时，需要保留足够精确的低阶项，才能判断答案：

$$\log(n!)=n\log n-(\log e)n+O(\log n)$$
$$\Longrightarrow\quad n\log n-\log(n!)=(\log e)n+O(\log n)=\Theta(n).$$

第一章作业：第 4 题

1. 证明对任意正实数 a 和 b，$a^n = O(b^n)$ 当且仅当 $a \leq b$。
2. 给定 $T(1) = 1$ 和 $T(n) = 2T(\lfloor n/2 \rfloor) + n$，证明 $T(n) = O(n \log n)$。

第一章作业：第 4 题（1）

1. 证明对任意正实数 a 和 b，$a^n = O(b^n)$ 当且仅当 $a \leq b$。

解

- 充分性：若 $a \leq b$，则对所有 $n \geq 1$，取常数 $C = 1$ 就有

$$a^n \leq Cb^n.$$

因此 $a^n = O(b^n)$。

- 必要性（反证法）：假设 $a > b$ 成立。由于 $a^n = O(b^n)$，一定存在常数 $C > 0$，使充分大的 n 满足

$$\left(\frac{a}{b}\right)^n \leq C.$$

如果 $a > b$，左边趋于无穷大，矛盾。所以 $a \leq b$。

第一章作业：第 4 题（2）

2. 给定 $T(1) = 1$ 和 $T(n) = 2T(\lfloor n/2 \rfloor) + n$，证明 $T(n) = O(n \log n)$。

解

直接证一个稍强的结论：对所有整数 $n \geq 1$，

$$T(n) \leq n(\log n + 1).$$

1. $n = 1$：$T(1) = 1 = 1(\log 1 + 1)$。

2. 假设所有小于 n 的正整数都满足结论，则

$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\leq 2\lfloor n/2 \rfloor(\log \lfloor n/2 \rfloor + 1) + n \\ &\leq n \log n + n. \end{aligned}$$

   * 最后一步用到 $2\lfloor n/2 \rfloor \leq n$、$\log \lfloor n/2 \rfloor + 1 \leq \log n$。

因此 $T(n) = O(n \log n)$。

第一章作业：第 4 题（2）

2. 给定 $T(1) = 1$ 和 $T(n) = 2T(\lfloor n/2 \rfloor) + n$，证明 $T(n) = O(n \log n)$。

错解 1

假设对所有正整数 $1 \leq k < n$，有 $T(k) \leq Ck \log k$，然后代入：

$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\leq 2C\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \\ &\leq Cn \log n - Cn + n \\ &\leq Cn \log n. \end{aligned}$$

第一章作业：第 4 题（2）

2. 给定 $T(1) = 1$ 和 $T(n) = 2T(\lfloor n/2 \rfloor) + n$，证明 $T(n) = O(n \log n)$。

错解 1

假设对所有正整数 $1 \le k < n$，有 $T(k) \le Ck \log k$，然后代入：

$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\le 2C\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \\ &\le Cn \log n - Cn + n \\ &\le Cn \log n. \end{aligned}$$

错点：$k = 1$ 时，$T(1) = 1$，但假设中 $k \log k = 1 \log 1 = 0$，不满足。

需要先补充上 $n = 2, 3$ 的基础情况，然后再用归纳法处理 $n \ge 4$ 的情况。

第一章作业：第 4 题（2）

2. 给定 $T(1) = 1$ 和 $T(n) = 2T(\lfloor n/2 \rfloor) + n$，证明 $T(n) = O(n \log n)$。

错解 2

$$
\begin{aligned}
T(n) &= 2T(n/2) + n \\
&= 4T(n/4) + 2n \\
&= \cdots \\
&= nT(1) + n \log n \\
&= n + n \log n.
\end{aligned}
$$

第一章作业：第 4 题（2）

2. 给定 $T(1) = 1$ 和 $T(n) = 2T(\lfloor n/2 \rfloor) + n$，证明 $T(n) = O(n \log n)$。

错解 2:

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 4T(n/4) + 2n \\ &= \cdots \\ &= nT(1) + n \log n \\ &= n + n \log n. \end{aligned}$$

错点：只适用于 n 是 2 的幂的情形，尚未覆盖一般整数。

第一章补充练习题

下列函数 $\mathrm{pre}(n)$ 可求出 2 至 n 中的所有素数，请分析其时间复杂度：

```
int primes[N], total;
bool not_prime[N];
void pre(int n) {
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i])
            primes[total++] = i;
        for (int j = 0; j < total; j++) {
            int p_j = primes[j];
            if (i * p_j > n)
                break;
            not_prime[i * p_j] = true;
            if (i % p_j == 0)
                break;
        }
    }
}
```

第一章补充练习题

```
int primes[N], total;
bool not_prime[N];
void pre(int n) {
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i])
            primes[total++] = i;
        for (int j = 0; j < total; j++) {
            int p_j = primes[j];
            if (i * p_j > n)
                break;
            not_prime[i * p_j] = true;
            if (i % p_j == 0)
                break;
        }
    }
}
```

其中：

- `primes[N]` 存储找到的素数，`total` 为当前找到的素数个数。
- `not_prime` 用于判断 i 是否为素数。
- 外层循环：遍历 2 至 n 的所有整数。
- 内层循环：用已经找到的素数来标记合数。

第一章补充练习题

```
int primes[N], total;
bool not_prime[N];
void pre(int n) {
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i])
            primes[total++] = i;
        for (int j = 0; j < total; j++) {
            int p_j = primes[j];
            if (i * p_j > n)
                break;
            not_prime[i * p_j] = true;
            if (i % p_j == 0)
                break;
        }
    }
}
```

其中：

- `primes[N]` 存储找到的素数，`total` 为当前找到的素数个数。
- `not_prime` 用于判断 i 是否为素数。
- 外层循环：遍历 2 至 n 的所有整数。
- 内层循环：用已经找到的素数来标记合数。

内层按从小到大的顺序枚举质数，遇到 i 的最小质因子时，第二个 `break` 跳出循环。

因此每次标记合数 $i * p_j$ 时，都有 p_j 小于等于 i 的最小质因数，所以 p_j 一定是 $i * p_j$ 的最小质因数。

对于每个合数 x，标记它的 p_j 和 $i = x/p_j$ 唯一确定，所以每个合数至多被标记一次。

每个质数也只会被加入数组一次，总的时间复杂度为 $O(n)$。

第一章补充练习题

```
int primes[N], total;
bool not_prime[N];
void pre(int n) {
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i])
            primes[total++] = i;
        for (int j = 0; j < total; j++) {
            int p_j = primes[j];
            if (i * p_j > n)
                break;
            not_prime[i * p_j] = true;
            if (i % p_j == 0)
                break;
        }
    }
}
```

为什么每个合数一定都能被标记？

对于任意合数 $x \leq n$，设其最小质因数为 p，令 $i = x/p$。

由于 p 是 x 的最小质因子，所以 $i = x/p \geq p$，故外层遍历到 i 时，p 一定已经在 primes 数组中。

内层遍历到 p 时，对于之前枚举的质数 $q < p$，又有 $iq < ip = x \leq n$ 且 $q \nmid i$，所以不会因为越界或者整除 i 退出内层循环，因此一定能枚举到 p，并标记 x。

第一章补充练习题

```
int primes[N], total;
bool not_prime[N];
void pre(int n) {
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i])
            primes[total++] = i;
        for (int j = 0; j < total; j++) {
            int p_j = primes[j];
            if (i * p_j > n)
                break;
            not_prime[i * p_j] = true;
            // if (i % p_j == 0)
            //    break;
        }
    }
}
```

第一章补充练习题

```
int primes[N], total;
bool not_prime[N];
void pre(int n) {
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i])
            primes[total++] = i;
        for (int j = 0; j < total; j++) {
            int p_j = primes[j];
            if (i * p_j > n)
                break;
            not_prime[i * p_j] = true;
            // if (i % p_j == 0)
            //     break;
        }
    }
}
```

如果去掉内层循环的退出条件 `if (i % p_j == 0) break;`，则每个合数可能会被不同的质数标记多次。

一个质数在外层 $i = p$ 时会进入质数表，之后会参与标记

$$p^2, p(p+1), p(p+2), \ldots, p(\lfloor \frac{n}{p} \rfloor)$$

总标记次数为

$$\sum_{\substack{p \le \sqrt{n} \\ p \text{ is prime}}} (\lfloor \frac{n}{p} \rfloor - p + 1) = \Theta(n \log \log n).$$

这里用到了质数倒数和为 $\Theta(\log \log n)$ 的结论。

Outline

1. 第一章作业
2. 第二章作业
3. 第三章作业
4. OJ 编程题

第二章作业：第 1 题

给定一个**带头结点**的单链表 L，设计一个算法将链表中的所有元素**原地逆置**，要求只使用 $O(1)$ 的额外空间。

第二章作业：第 1 题

给定一个**带头结点**的单链表 L，设计一个算法将链表中的所有元素**原地逆置**，要求只使用 $O(1)$ 的额外空间。

解 保留头结点，依次翻转数据结点的 next。

```
void reverse(Node *L) {
    Node *prev = NULL, *cur = L->next;
    while (cur != NULL) {
        Node *next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    L->next = prev;
}
```

正确性：设原结点依次为 $v_1, \ldots, v_n$。第 i 轮将 v_i 接到已逆置部分的最前面，得到 $v_i \to \cdots \to v_1 \to \mathrm{NULL}$。共执行 n 轮，得到完整逆序链表，再令头结点指向 v_n 即可。

时间 $O(n)$，额外空间 $O(1)$。

第二章作业：第 1 题

给定一个**带头结点**的单链表 L，设计一个算法将链表中的所有元素**原地逆置**，要求只使用 $O(1)$ 的额外空间。

错解 从前往后取结点，每次插到头结点之后。

```
void reverse(Node *head) {
    Node *hd = head->next, *u = hd, *v = u->next;
    while (v != NULL) {
        u->next = (u == hd) ? NULL : head->next;
        head->next = u;
        u = v;
        v = u->next;
    }
}
```

第二章作业：第 1 题

给定一个**带头结点**的单链表 L，设计一个算法将链表中的所有元素**原地逆置**，要求只使用 $O(1)$ 的额外空间。

错解 从前往后取结点，每次插到头结点之后。

```
void reverse(Node *head) {
    Node *hd = head->next, *u = hd, *v = u->next;
    while (v != NULL) {
        u->next = (u == hd) ? NULL : head->next;
        head->next = u;
        u = v;
        v = u->next;
    }
}
```

错点：处理的是 u，却用 v 判断是否继续。u 到达尾结点时，v == NULL，尾结点被漏掉。

第二章作业：第 1 题

给定一个**带头结点**的单链表 L，设计一个算法将链表中的所有元素**原地逆置**，要求只使用 $O(1)$ 的额外空间。

解法二：头插法 保存原首结点，将 head->next 置空，再依次把原链表结点插到头结点之后。

```
void reverse(Node *head) {
    Node *u = head->next;
    head->next = NULL;
    while (u != NULL) {
        Node *v = u->next;
        u->next = head->next;
        head->next = u;
        u = v;
    }
}
```

正确性：每次头插后，已处理部分始终逆序；`u == NULL` 时，所有结点均已接入。
时间 $O(n)$，额外空间 $O(1)$。

第二章作业：第 2 题

给定一个**不带头结点**的单链表 L，其中 head 指向第一个数据结点，设计算法找到其中间节点。若节点数为偶数，规定返回两个中间节点中的**后一个**。要求只遍历链表一次。

第二章作业：第 2 题

给定一个**不带头结点**的单链表 L，其中 head 指向第一个数据结点，设计算法找到其中间节点。若节点数为偶数，规定返回两个中间节点中的**后一个**。要求只遍历链表一次。

解 快慢指针同时出发，分别每次走两步、一步。

```
Node* findMiddle(Node* head) {
    Node *slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

正确性：设链表长度为 n。循环结束时，fast 到达尾结点（n 为奇数）或 NULL（n 为偶数），共循环 $\lfloor n/2 \rfloor$ 次。因此 slow 走了 $\lfloor n/2 \rfloor$ 步，指向第 $\lfloor n/2 \rfloor + 1$ 个结点，恰好是中间结点；偶数时取后一个。

时间 $O(n)$，额外空间 $O(1)$。

第二章作业：第 2 题

给定一个**不带头结点**的单链表 L，其中 head 指向第一个数据结点，设计算法找到其中间节点。若节点数为偶数，规定返回两个中间节点中的**后一个**。要求只遍历链表一次。

错解 遍历时把结点指针存入数组，再取中间位置。

```
Node *findMiddle(Node *head, Node **nodes) {
    int len = 0;
    Node *cur = head;
    while (cur->next != NULL) {
        nodes[len++] = cur;
        cur = cur->next;
    }
    return nodes[(len + 1) / 2];
}
```

第二章作业：第 2 题

给定一个**不带头结点**的单链表 L，其中 head 指向第一个数据结点，设计算法找到其中间节点。若节点数为偶数，规定返回两个中间节点中的**后一个**。要求只遍历链表一次。

错解 遍历时把结点指针存入数组，再取中间位置。

```
Node *findMiddle(Node *head, Node **nodes) {
    int len = 0;
    Node *cur = head;
    while (cur->next != NULL) {
        nodes[len++] = cur;
        cur = cur->next;
    }
    return nodes[(len + 1) / 2];
}
```

错点：循环条件导致尾结点未被存入。长度为 1 时，没有存入任何结点，却读取 nodes[0]；长度为 2 时，只存入 nodes[0]，却读取 nodes[1]。

使用数组本身并未违反题目要求，但推荐同学们思考时空复杂度更低的解法。

第二章作业：第 3 题

给定两个**无环单链表** L_1, L_2，请设计算法判断它们是否相交；若相交，找出它们的**第一个公共节点**。要求 $O(1)$ 额外空间。

第二章作业：第 3 题

给定两个**无环单链表** L_1, L_2，请设计算法判断它们是否相交；若相交，找出它们的**第一个公共节点**。要求 $O(1)$ 额外空间。

解法一 将两条链表的剩余长度对齐。

1. 遍历两表，得到长度 m, n。
2. 长表的指针先走 $|m-n|$ 步。
3. 两个指针同步向后走，第一次相等时返回该结点；同时到达 `NULL` 则无交点。

正确性：相交后两表共用同一段后缀。剩余长度相同，故两指针同时进入公共后缀，第一次相等就是第一个交点。

比较结点地址，不是数据值。时间 $O(m+n)$，额外空间 $O(1)$。

第二章作业：第 3 题

给定两个**无环单链表** L_1, L_2，请设计算法判断它们是否相交；若相交，找出它们的**第一个公共节点**。要求 $O(1)$ 额外空间。

解法二 两个指针分别从两表出发，走到 NULL 后换到另一表。

```
Node *firstCommon(Node *L1, Node *L2) {
    Node *p = L1, *q = L2;
    while (p != q) {
        p = (p == NULL) ? L2 : p->next;
        q = (q == NULL) ? L1 : q->next;
    }
    return p;
}
```

正确性：设独有前缀长为 a, b，公共后缀长为 c。若未提前相遇，两指针从各自起点出发，均在 $a + b + c + 1$ 次循环之后到达交点；无交点则最终同时到达 NULL。

时间 $O(m + n)$，额外空间 $O(1)$。

第二章作业：第 3 题

给定两个**无环单链表** L_1, L_2，请设计算法判断它们是否相交；若相交，找出它们的**第一个公共节点**。要求 $O(1)$ 额外空间。

错解 走到 NULL 立即换头，然后判断是否相等。

```
Node *firstCommon(Node *L1, Node *L2) {
    Node *p = L1, *q = L2;
    while (1) {
        p = p->next; q = q->next;
        if (p == NULL) p = L2;
        if (q == NULL) q = L1;
        if (p == q) return p;
    }
}
```

第二章作业：第 3 题

给定两个**无环单链表** L_1, L_2，请设计算法判断它们是否相交；若相交，找出它们的**第一个公共节点**。要求 $O(1)$ 额外空间。

错解 走到 NULL 立即换头，然后判断是否相等。

```
Node *firstCommon(Node *L1, Node *L2) {
    Node *p = L1, *q = L2;
    while (1) {
        p = p->next; q = q->next;
        if (p == NULL) p = L2;
        if (q == NULL) q = L1;
        if (p == q) return p;
    }
}
```

错点：两条等长、不相交的链表会不断重走，无法终止；若两条链从第一个结点开始就相交，也可能漏掉第一个交点。

应在循环入口判等，保留 NULL 状态。

Outline

1. 第一章作业
2. 第二章作业
3. 第三章作业
4. OJ 编程题

第三章作业：第 1 题

原栈只保存 `int`，不增加新容器，使 `push/pop/top/getMax` 均为 $O(1)$。

第三章作业：第 1 题

原栈只保存 `int`，不增加新容器，使 `push/pop/top/getMax` 均为 $O(1)$。

解 每个真实值后面存一个前缀最大值：

$$x_1, M_1,\ x_2, M_2,\ \ldots,\ x_k, M_k, \qquad M_i = \max(x_1, \ldots, x_i).$$

1. `push(x)`：先压真实值，再压更新后的最大值。
2. `pop()`：先弹辅助最大值，再弹真实值并返回。
3. `top()`：暂时取下辅助值，读取真实值后恢复原状。
4. `getMax()`：直接读取原栈顶。

每次弹出一对整数，上一组最大值自然恢复。各操作的存储顺序保持一致即可。

第三章作业：第 1 题

原栈只保存 int，不增加新容器，使 push/pop/top/getMax 均为 $O(1)$。

原有整数栈为 S。

```
void push(int x) {
    int m = x;
    if (!S.empty() && S.top() > m)
        m = S.top();
    S.push(x);
    S.push(m);
}

int pop() {
    S.pop(); // 移除辅助最大值
    int x = S.top();
    S.pop(); // 移除真实值
    return x;
}
```

```
int top() {
    int m = S.top();
    S.pop();
    int x = S.top();
    S.push(m); // 恢复辅助最大值
    return x;
}

int getMax() {
    return S.top();
}
```

第三章补充题

使用栈解决以下问题，并给出算法的时间复杂度：

- 给定一个长度为 n 的序列 $a_1, a_2, \ldots, a_n$，对于每个位置 $i < n$，求出最小的 $j > i$ 使得 $a_j > a_i$，若不存在，该位置的答案为 0。

样例 序列为 $3, 1, 3, 4, 3, 2, 1, 5, 6$，下标从 1 开始。

位置 i	1	2	3	4	5	6	7	8	9
a_i	3	1	3	4	3	2	1	5	6
答案 j	4	3	4	8	8	8	8	9	—

第三章补充题

使用栈解决以下问题，并给出算法的时间复杂度：

- 给定一个长度为 n 的序列 $a_1, a_2, \ldots, a_n$，对于每个位置 $i < n$，求出最小的 $j > i$ 使得 $a_j > a_i$，若不存在，该位置的答案为 0。

思路 很容易写出两重循环的暴力解法，其复杂度为 $O(n^2)$，但是我们发现这样会扫到很多没有必要扫的数字。

例如，在刚才的样例 $3, 1, 3, 4, 3, 2, 1, 5, 6$ 中，下标 4、5、6、7 都要向右找到下标 8 的 5，查找过程存在重复。能否在扫描到 5 时，一次性确定这些下标的答案？

注意到，这些位置对应的值按下标顺序单调不增。否则，前面的元素会先遇到其中一个更大的值，答案就不会都是下标 8。我们可以利用这个性质来优化算法。

第三章补充题

如何维护并利用单调不增的性质？

当新元素到来时，我们从保存的单调不增序列的最后一个数开始看：

1. 如果它小于新元素，说明新元素就是它右边第一个更大的数；
2. 如果它大于等于新元素，说明它的答案一定还在新元素的右边（或不存在）。

这提示我们用单调栈来维护当前还没有找到答案的元素位置。

第三章补充题

维护一个下标对应的元素从栈底到栈顶单调递减的栈，栈中存储的是还没有找到答案的位置下标。每次新元素 a_j 到来时：

- 如果栈空或栈顶元素 a_i 大于等于 a_j，压入 j。后者对应的情况为 a_i 的答案在 a_j 的右边，
- 如果栈顶元素 a_i 小于 a_j，则 a_j 就是 a_i 右边第一个更大的数，弹出栈顶并记录答案 j，重复比较新的栈顶元素，直到栈空或栈顶元素大于等于 a_j。

如果遍历完所有元素后，栈中仍有元素，说明并不能在它们右边找到比它们更大的元素，因此它们的答案为 0。

每个元素最多被压入和弹出栈各一次，因此时间复杂度为 $O(n)$。

第三章补充题

代码实现

```
for (int j = 1; j <= n; ++j) {
    while (!st.empty() && a[st.top()] < a[j]) { // 栈顶元素小于当前元素，弹出栈顶
        ans[st.top()] = j;
        st.pop();
    }
    st.push(j);
}
```

OJ：滑动窗口

给定长度为 n 的数组（$n \le 10^6$），一个大小为 k 的窗口从左向右滑动，每次移动一个位置。求每个窗口中的**最小值与最大值**。

- **输入**：第一行是 n, k，第二行是数组的 n 个数。
- **输出**：两行，依次为各窗口的最小值、各窗口的最大值。

样例输入

```
8 3
1 3 -1 -3 5 3 6 7
```

样例输出

```
-1 -3 -3 -3 3 3
3 3 5 5 6 7
```

时间限制：12000 ms　　内存限制：65536 kB

OJ：滑动窗口

给定长度为 n 的数组（$n \leq 10^6$），一个大小为 k 的窗口从左向右滑动，每次移动一个位置。求每个窗口中的**最小值与最大值**。

考虑对暴力解法进行优化（以求最小值为例）。

不难观察到，对于两个下标 $i < j$，如果 $a_i \geq a_j$，那么 a_i 不可能再对后续窗口的最小值产生贡献。这是因为窗口从左向右滑动，a_i 一定比 a_j 更早离开窗口。

所以，当新元素进入窗口时，可以淘汰其左边所有大于或等于它的候选元素。这样，剩余候选元素的值从左到右严格递增。

由于我们既需要从队首弹出离开窗口的元素，也需要从队尾弹出不满足单调性的元素，因此可以使用双端队列维护当前窗口的最小值。

OJ：滑动窗口

用双端队列维护最小值候选

具体地，队列存储元素的下标，并满足：

- 下标从队首到队尾严格递增；
- 对应的元素值从队首到队尾严格递增。

因此，队首元素对应的值就是当前窗口的最小值。

该队列支持从队尾插入、弹出队首元素、弹出队尾元素、获取队首队尾的值的操作，且复杂度均为 $O(1)$。

每个元素最多被插入和弹出队列各一次，因此时间复杂度为 $O(n)$。

OJ：滑动窗口

```
deque<int> q; // C++ STL 库中的双端队列
// 最小值
for (int i = 1; i <= n; i++) {
    while (!q.empty() && q.front() <= i - k) q.pop_front(); // 队首过期
    while (!q.empty() && a[q.back()] >= a[i]) q.pop_back(); // 队尾淘汰
    q.push_back(i);
    if (i >= k) cout << a[q.front()] << " \n"[i == n];
}
// 最大值
q.clear(); // 清空队列
for (int i = 1; i <= n; i++) {
    while (!q.empty() && q.front() <= i - k) q.pop_front(); // 队首过期
    while (!q.empty() && a[q.back()] <= a[i]) q.pop_back(); // 队尾淘汰
    q.push_back(i);
    if (i >= k) cout << a[q.front()] << " \n"[i == n];
}
```

OJ：滑动窗口

```
int q[N]; // 用数组模拟双端队列
int head = 1, tail = 1; // 空队列: head == tail
// 最小值
for (int i = 1; i <= n; i++) {
    while (head < tail && q[head] <= i - k) head++; // 队首过期
    while (head < tail && a[q[tail - 1]] >= a[i]) tail--; // 队尾淘汰
    q[tail++] = i;
    if (i >= k) cout << a[q[head]] << " \n"[i == n];
}
// 最大值
head = tail = 1; // 清空队列
for (int i = 1; i <= n; i++) {
    while (head < tail && q[head] <= i - k) head++;
    while (head < tail && a[q[tail - 1]] <= a[i]) tail--;
    q[tail++] = i;
    if (i >= k) cout << a[q[head]] << " \n"[i == n];
}
```

OJ：堆栈基本操作

将 $1, 2, \ldots, n$ 按顺序入栈，入栈和出栈操作可以交替进行。给定一个长度为 n 的目标序列 $\{a_i\}_{i=1}^{n}$，判断它能否成为最终的出栈序列。

- **输入**：第一行是 n，第二行是目标序列。
- **输出**：若可行，依次输出操作，每个操作一行：`PUSH x` 或 `POP x`；否则只输出 `NO`。
- **注意**：目标序列中可能出现不在 $1, \ldots, n$ 范围内的数。

样例输入

```
7
4 5 3 6 2 7 1
```

样例可行：先依次压入 $1, 2, 3, 4$，再弹出 4；随后压入 5，弹出 $5, 3$，继续完成其余操作。

时间限制：1000 ms　　内存限制：65535 kB

OJ：堆栈基本操作

首先如果目标序列中存在 $1, \ldots, n$ 范围之外的数，则直接输出 `NO`。

依次考虑目标序列中的每个数字 a_i：

- 如果此时 a_i 还没有入栈，从下一个尚未入栈的数开始，依次压入直到 a_i。
- 如果此时 a_i 已经在栈顶，则弹出它。
- 否则无解，输出 `NO`。

每个元素至多入栈、出栈各一次，时间 $O(n)$，空间 $O(n)$。

正确性：

- 充分性：如果算法模拟成功，算法记录的操作就是一组合法方案。
- 必要性：如果目标序列合法，那么算法一定可以通过模拟得到对应的操作序列。

OJ：堆栈基本操作

为什么模拟失败，目标序列就一定不合法？

分析刚才输出无解的情况：

1. a_i 仍在栈内，但不在栈顶：此时栈顶元素为 $y \neq a_i$。若要弹出 a_i，就必须先弹出 y，与下一个目标值是 a_i 矛盾。

2. a_i 已经出过栈：每个数只能入栈、出栈各一次，若已经出过栈，就不可能再出栈。

此时是否存在另外一种操作方案使得该目标序列合法？

此时是否存在另外一种操作方案使得该目标序列合法？

假设存在一种能够完成目标序列的方案，将它与算法从初始空栈开始比较。假设下一项输出是 x：

1. **若栈顶是 x**：成功方案必须弹出 x。若先压入其他数，就必须先弹出那个数，违反目标顺序。
2. **若栈空或栈顶不是 x**：成功方案不能弹栈，只能压入下一个尚未入栈的数；若没有剩余输入，则无法成功。

因此，成功方案不可能在任何一步与算法作出不同选择。若算法失败，假设中的成功方案也无法完成，矛盾。

由此我们证明了算法模拟失败时目标序列一定不合法。